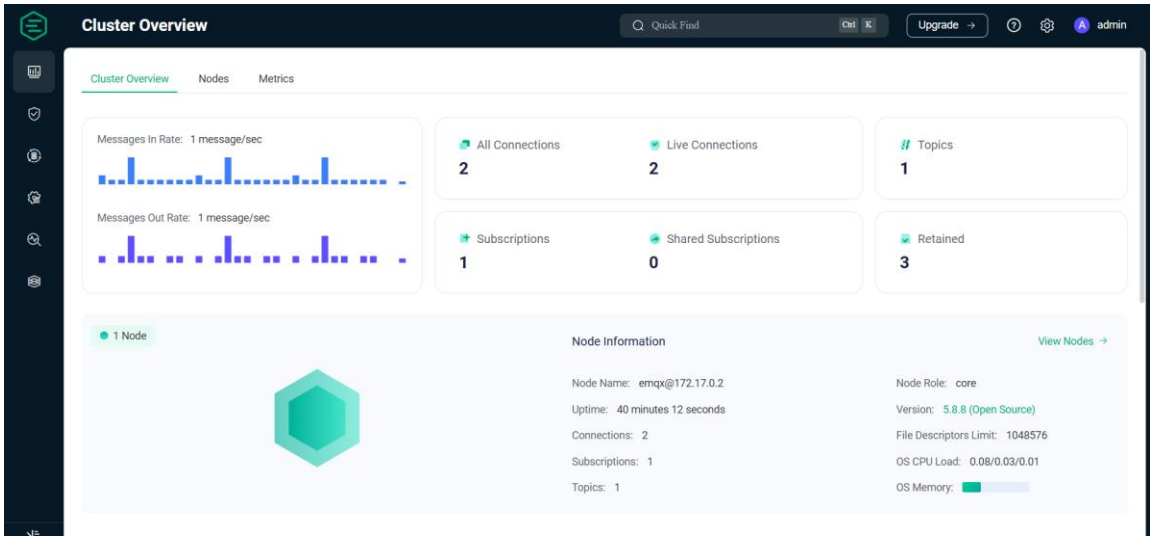
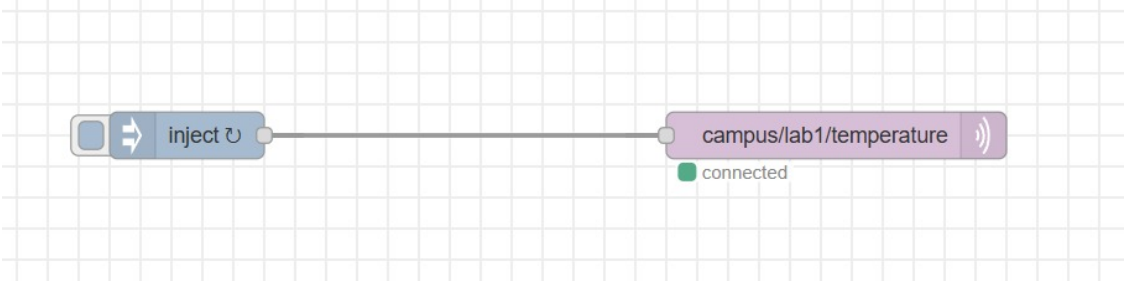
MAHENDRAN A/L THILAGAMANY B122510490

EXERCISE 4: MQTT Publish/Subscribe

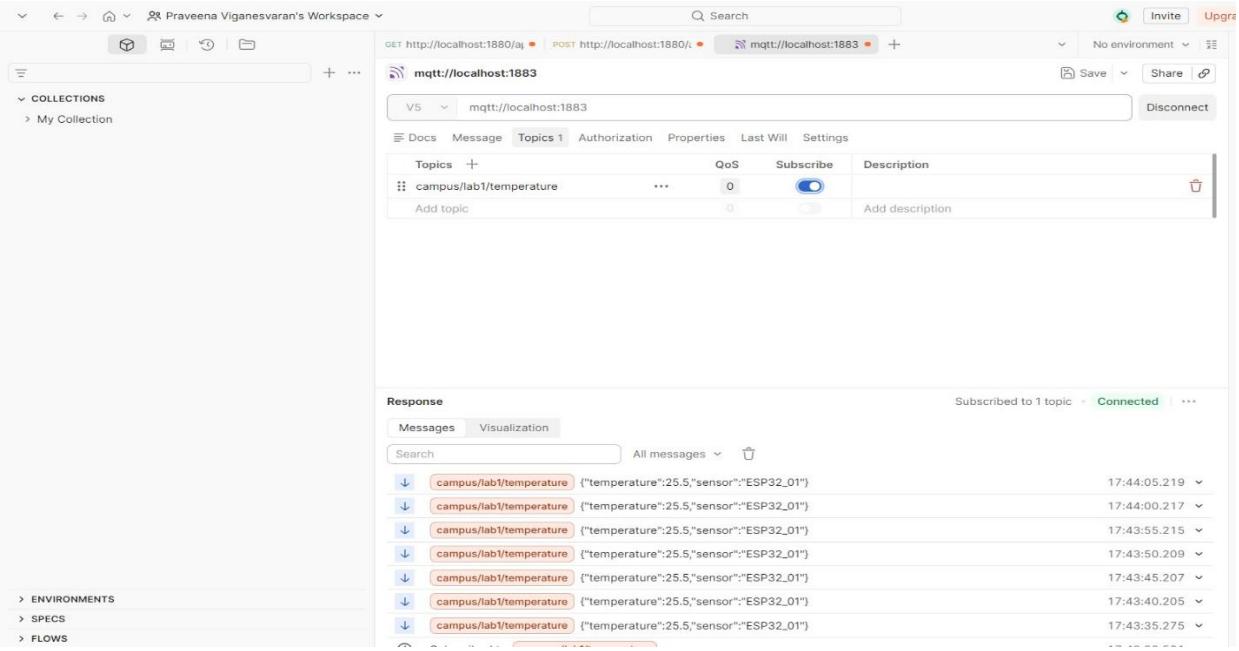
Step 1: Initialize the EMQX MQTT Broker via Docker



Step 2: Configure Node-RED as an MQTT Publisher

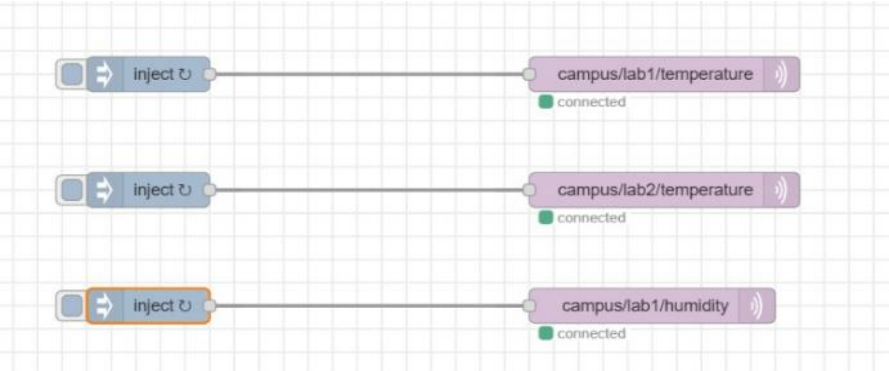


Step 3: Configure Postman as an MQTT Subscriber

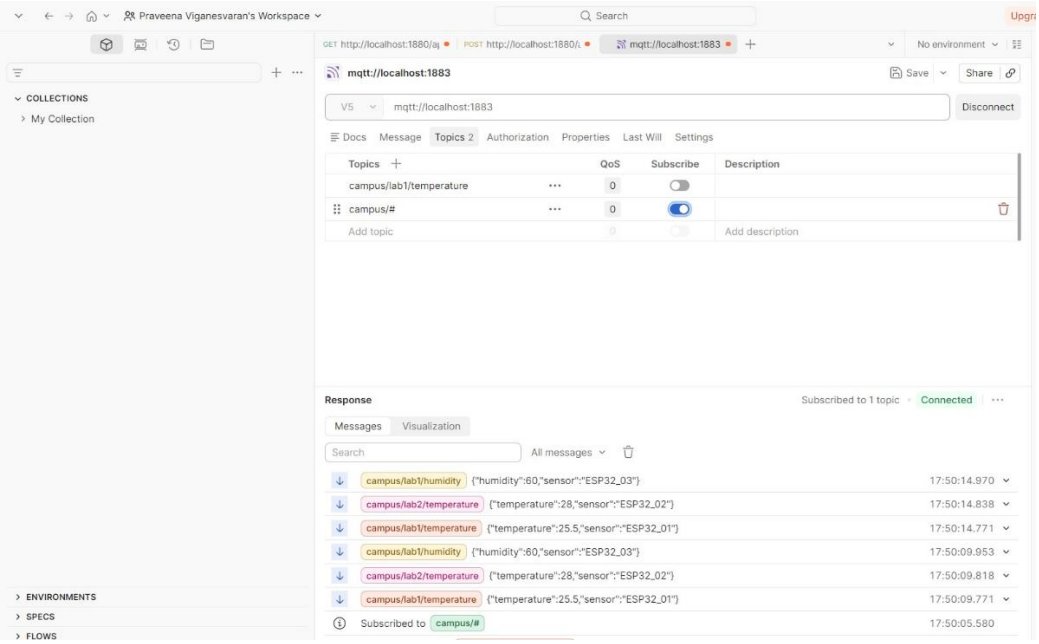


Step 4: Advanced Topic Hierarchies and Wildcards

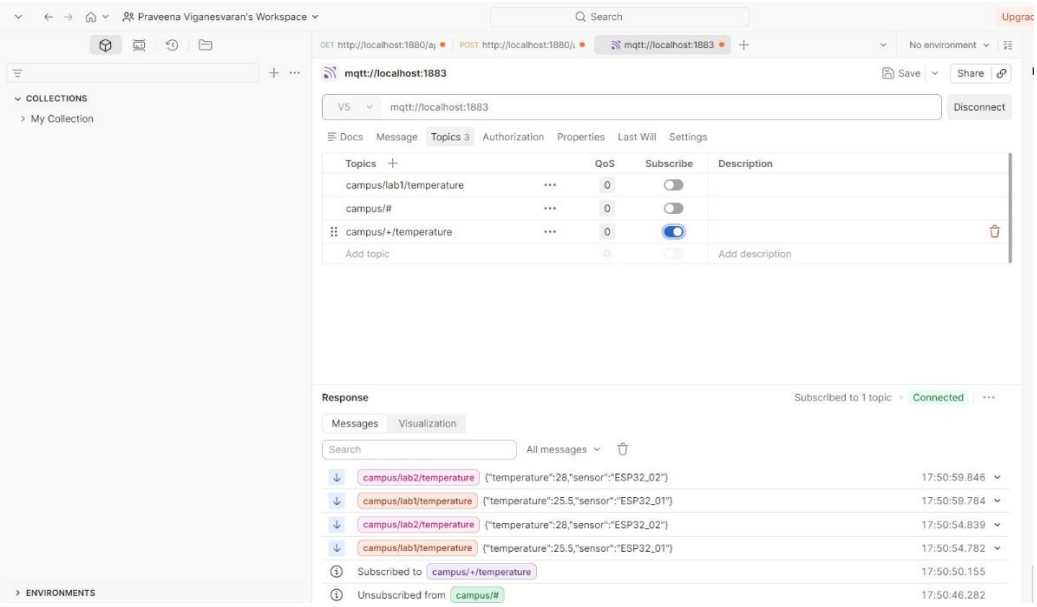
1. Modify Node-RED Publishers



2. Experiment with the # (Multi-Level) Wildcard



3. Experiment with the + (Single-Level) Wildcard



Step 5: Discussion & Architectural Analysis

Question 1: The Role of the Broker

In Week 3, you sent data directly from Postman (Client) to Node-RED (Server) using HTTP. Today, both Node-RED and Postman connected to the EMQX Broker instead. Explain the architectural role of the MQTT broker. What is the advantage of MQTT compared to the HTTP protocol?

In the MQTT protocol, the Broker acts as the central hub or post office for all messages. Unlike the direct Client-Server model used in HTTP, where the sender and receiver must be aware of each other's IP addresses and be online at the same time, the MQTT broker handles decoupling.

Architectural Role: The broker receives all incoming messages from publishers and then filters and distributes them to the correct subscribers based on specific topics (like campus/lab1/temperature).

Advantages over HTTP:-

- Lightweight: MQTT has a much smaller packet header (as low as 2 bytes) compared to the heavy text-based headers of HTTP, making it ideal for low-bandwidth IoT devices.
- Many-to-Many Communication: One publisher can send data to hundreds of subscribers simultaneously through the broker, whereas HTTP is typically a one-to-one request/response.
- Reduced Power Consumption: Devices don't need to keep a constant, heavy connection open or poll the server; they simply wait for the broker to push data to them.

Question 2: Publish vs. Subscribe

Define the specific roles of a "Publisher" and a "Subscriber" in the MQTT protocol. Can a single IoT device (like an ESP32) be both a publisher and a subscriber simultaneously? Provide a real-world example of why a device would need to do both.

- **Publisher:** The device that sends data to the broker. It categorizes the data into a "topic." (e.g., Node-RED sending a temperature value).
- **Subscriber:** The device that expresses interest in a specific topic. It "listens" to the broker and receives any data published to that topic. (e.g., Postman or a dashboard waiting for updates).

Can a device be both?

Yes. Most IoT devices act as both simultaneously.

- Real-World Example: Consider a Smart Thermostat.
- It acts as a Publisher to send the current room temperature to your phone.
- It acts as a Subscriber to listen for "commands" from your phone (e.g., "Set temperature to 22°C") so it knows when to turn the AC on or off.

Question 3: Client vs. Server

In an HTTP REST API, the server must be running and listening before the client can send a POST request. In MQTT, what happens if Postman (the subscribing client) disconnects for 10 minutes while Node-RED (the publishing client) continues to send data? How does the broker handle this?

In HTTP, if the server is down or the client isn't listening, the data is usually lost or the request fails immediately. In MQTT, the broker can provide more flexibility:

1. **Standard Behaviour:** By default, if a subscriber (Postman) is disconnected, any messages sent by the publisher (Node-RED) during that time are simply discarded by the broker.
2. **Persistent Sessions (QoS):** If the client connects with a Clean Session = False flag and uses Quality of Service (QoS) level 1 or 2, the broker will store the missed messages. Once Postman reconnects, the broker "catches it up" by delivering the missed data.
3. **Retained Messages:** If Node-RED publishes a message with the "Retain" flag set to true, the broker will keep the last known good value for that topic. When Postman reconnects after 10 minutes, it will immediately receive that last saved value, so it isn't starting with blank data.